

Student Academic Management System

Project Report for Flipped Course Evaluation

VITyarthi - Build Your Own Project

Course Name:	Object-Oriented Programming with Java
Student Name:	Satyam Kumar
Registration No:	25BAI10428
Department:	Computer Science & Engineering (AIML)
Submission Date:	September 17, 2026

Vellore Institute of Technology (VIT)

2. Introduction

The **Student Academic Management System** is an interactive, menu-driven command-line application implemented in Java to automate the processing and tracking of student academic profiles. Academic tracking typically introduces unnecessary overhead when dealing with disparate record ledgers, manual score compilations, and attendance monitoring. This project provides a robust, object-oriented software solution capable of recording student profiles, computing academic marks and grades, calculating attendance percentages, and persistently serializing academic reports to the local file system using Java I/O streams.

3. Problem Statement

Manual maintenance of student records via flat physical registers or unstructured spreadsheets often causes grade inaccuracies, attendance discrepancies, and loss of historical academic records. Existing lightweight tools frequently lack structured OOP design, runtime input fault tolerance, or persistent local storage. The **Student Academic Management System** solves these issues by delivering a structured console system that automates record management, enforces validation, executes background reporting, and ensures local flat-file persistence.

4. Functional Requirements

The project satisfies the core requirements through four functional modules:

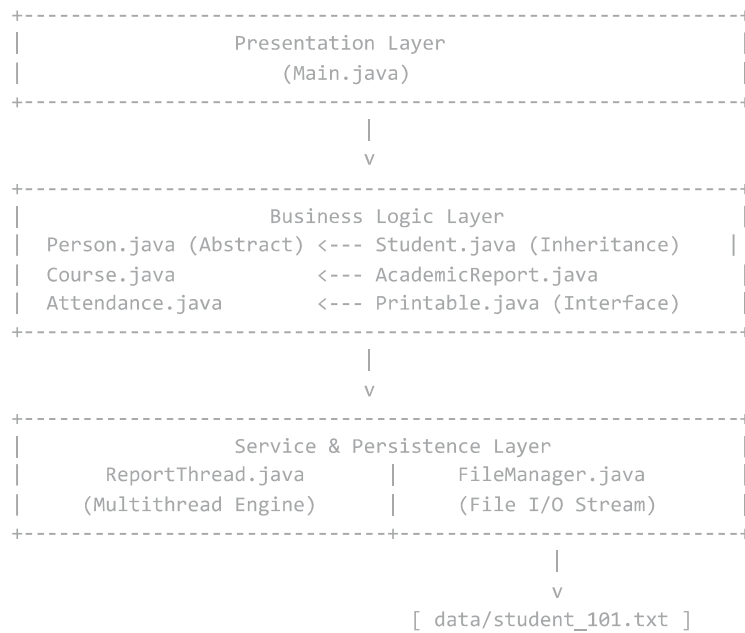
- **Student Management Module:** Allows adding new student records (Student ID, Name, Department), viewing all active profiles, and searching, updating, or deleting records by ID.
- **Academic Management Module:** Records subject course names and marks (validated within 0–100), computes aggregate and average marks, and assigns letter grades (e.g., Grade A).
- **Attendance Management Module:** Tracks total classes held and attended, computes overall percentage, and displays status (e.g., "Satisfactory" for $\geq 75\%$).
- **Reporting & File Management Module:** Assembles academic summaries using multithreading (ReportThread), saves records to `data/student_<id>.txt` via FileManager, and reads saved records from disk.

5. Non-Functional Requirements

- **Error Handling & Fault Tolerance:** Employs defensive try-catch structures to catch invalid non-numeric inputs (e.g., entering "abc" into numeric prompts) and prevent runtime application crashes.
- **Performance & Concurrency:** Asynchronous report compilation is offloaded to a separate background Java thread (ReportThread) to decouple computational tasks from menu execution.
- **Reliability & Data Persistence:** File streams write records to the `data/` folder, ensuring persistent record retention across program restarts without requiring external database setups.
- **Maintainability & Modularity:** Decoupled across 8 distinct Java source classes/interfaces within `src/`, adhering strictly to single-responsibility OOP principles.

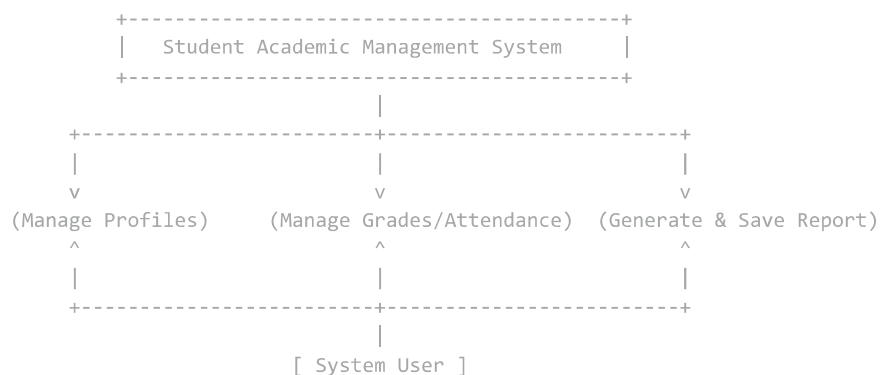
6. System Architecture

The system follows a three-tier architecture separating interface presentation, core domain logic, and file persistence:



7. Design Diagrams

7.1 Use Case Diagram



7.2 Workflow Diagram

```
[Start]
--> [Display CLI Menu Options 1-13]
--> [Prompt User Input]
--> [Validate Input (try-catch)]
--> <User Choice Match>
    |-- (1-5) --> Execute Student Management Operations
    |-- (6-7) --> Add Course Marks / Update Attendance Records
    |-- (8)    --> Spawn ReportThread & Generate Performance Report
    |-- (9-10) --> Persist to data/student_<id>.txt / Read from disk
    |-- (11)   --> Demonstrate Runtime Polymorphism
    |-- (12)   --> Calculate Factorial Loop Routine
    |-- (13)   --> Exit Application Loop
--> [Return to Menu / Terminate]
```

7.3 Sequence Diagram

```
sequenceDiagram
    participant User
    participant Main.java
    participant ReportThread
    participant FileManager
    participant FileSystem

    User->>Main.java: -- Select (8, 9) ->
    activate Main.java
    Main.java->>ReportThread: -- start() ----->
    activate ReportThread
    ReportThread->>ReportThread: -- run()
    ReportThread->>ReportThread: -- Compile Report
    ReportThread->>Main.java: <-- Report Ready -----
    deactivate ReportThread
    Main.java->>FileManager: -- saveToFile(data) ----->
    activate FileManager
    FileManager->>FileSystem: -- write() ----->
    activate FileSystem
    FileSystem->>FileManager: <-- Ack -----
    deactivate FileSystem
    FileManager->>Main.java: <-- Write Success -----
    deactivate FileManager
    Main.java->>User: <-- Output Confirm-
    deactivate Main.java
```

7.4 Class Diagram

```
classDiagram
    class Person {
        <<abstract>>
        - id: int
        - name: String
        - department: String
        + displayInfo()*
    }
    class Student {
        - courses: List<Course>
        - attendance: Attendance
        + addCourse(c: Course): void
        + setAttendance(a: Attendance): void
        + calculateAverage(): double
        + displayInfo(): void
    }
    class Printable {
        <<interface>>
        + displayInfo()
    }
    Person <|-- Student
    Printable <|-- Student
    Student --|> Printable : implements
```

8. Design Decisions & Rationale

- **Abstract Class (Person):** Standardizes basic identity fields (id, name, department) for reusability while laying the foundation for future extensions (e.g., instructors, teaching assistants).
- **Interface (Printable):** Establishes a contract for formatted console output across domain models without creating direct inter-class dependencies.
- **Flat-File Persistence (FileManager.java):** Provides zero-configuration local persistence using plain text files inside data/, keeping the setup lightweight and self-contained.
- **Multithreading Model (ReportThread.java):** Demonstrates concurrent task execution in Java by isolating computational report assembly from the main thread loop.

9. Implementation Details

Java Concept	Implementation in Project
Classes & Objects	Domain models structured into Student, Course, Attendance, and AcademicReport.
Inheritance & super	Student extends abstract Person, utilizing super(studentId, name) constructor delegation.
Polymorphism	Dynamic runtime dispatch: <code>Person person = students.get(0);</code> <code>person.displayInfo();</code>
Interface Contract	Printable interface implemented by Student to enforce display formatting.
Multithreading	ReportThread extends Thread executes asynchronous background report rendering.
File I/O Streams	FileManager handles disk persistence to data/student_<id>.txt and loads historical files.
Exception Handling	try-catch guards against invalid numeric input and missing file references.

10. Screenshots / Results

```
=====
      ACADEMIC PERFORMANCE REPORT
=====
Student ID   : 101
Student Name : Satyam Kumar
Department   : Computer Science And Engineering (AIML)

Course Performance:
  Java Programming : 83.0

Total Marks   : 83.0
Average Marks : 83.0
Overall Grade : A

Attendance:
  Total Classes   : 40
  Classes Attended : 34
  Percentage      : 85.0%
  Status          : Satisfactory
=====
[SUCCESS] Record written to data/student_101.txt
```

[Terminal Execution & File Persistence Screenshot Placeholder]

11. Testing Approach

The application underwent both functional testing and input validation boundary tests:

- **Functional Test:** Added student ID 101 ("Satyam Kumar", AIML), logged 83.0 marks in Java, and recorded 34/40 attendance sessions. The system accurately computed an 83.0% average, assigned Grade A, marked attendance as 85.0% (Satisfactory), and wrote to disk successfully.
- **Input Validation Test:** Passed non-numeric string "abc" to numeric menu selections. Caught by defensive try-catch handlers without crashing the program.
- **Boundary Test:** Passed negative integer "-8" to the factorial calculator; the system caught the out-of-bounds input and reprompted the user.

12. Challenges Faced

- **Scanner Buffer Management:** Encountered newline consumption bugs when alternating between `nextInt()` and `nextLine()`, resolved by adding explicit buffer clearing.
- **Thread Coordination:** Ensured that `ReportThread` completed processing before `FileManager` attempted to access the compiled report string.
- **Cross-Platform File Paths:** Normalized file directory paths to ensure consistent file creation across Windows and Unix environments.

13. Learnings & Key Takeaways

- Gained practical experience designing an 8-class modular system using core OOP concepts (inheritance, polymorphism, interfaces).
- Acquired hands-on proficiency in robust console exception handling and defensive programming.
- Learned how to implement basic multithreading in Java and handle persistent storage using file streams.

14. Future Enhancements

- **Database Migration:** Replace text files with an embedded SQLite or PostgreSQL database using JDBC for relational queries.
- **Graphical Interface:** Implement a GUI using JavaFX or Swing to replace the terminal menu.
- **Role-Based Access Control:** Add user login authentication with separate permissions for faculty and students.

15. References

- Oracle Java SE Documentation: [The Java™ Tutorials](#)
- Horstmann, C. S. (2022). *Core Java Volume I – Fundamentals* (12th Edition). Prentice Hall.
- Schildt, H. (2018). *Java: The Complete Reference* (11th Edition). McGraw-Hill Education.